ECOS DEL GRIMORIO

Cómo hice el juego en Unity

Mis notas para explicar el proyecto



Pérez Alcántara Alejandro · 10° "D"
Creación de Videojuegos · Universidad Tecnológica de Puebla
Unity 6 · C# · Versión 1.0

Antes de empezar

Estas son mis notas para poder explicar cómo hice Ecos del Grimorio si el profe o algún compañero me pregunta. Cada tema empieza con la explicación corta (lo que diría en voz alta) y luego viene el detalle, con el nombre del script donde está, para poder enseñarlo en el código si me lo piden.

La idea general en una frase: el juego está hecho en Unity 6 con C#. En lugar de armar las escenas a mano, programé una herramienta de editor (CrearJuego.cs) que crea todos los prefabs y las nueve escenas desde código; los datos de hechizos, ítems y enemigos están en ScriptableObjects, los enemigos usan herencia y la música la generé con Python.

Tema	Script principal
Construir niveles, prefabs y fondos	Assets/Editor/CrearJuego.cs y EditorHelpers.cs
Movimiento de Lira	Scripts/Player/PlayerController.cs
Hechizos y combos	Scripts/Player/SpellCaster.cs, SpellProjectile.cs y Data/SpellData.cs
Vida y daño	Scripts/Managers/Health.cs (interfaz IDamageable)
Enemigos y jefes	Scripts/Enemigos/EnemyBase.cs, BossController.cs y sus hijos
Armas de los enemigos	Scripts/Enemigos/ArmaEnemigo.cs
Habilidades del entorno	Scripts/Efectos/Rompible.cs, Empujable.cs, PuertaRunica.cs, CorrienteViento.cs
Truco (Modo Archimaga)	Scripts/Player/ModoArchimaga.cs
Música y sonidos	Scripts/Audio/AudioManager.cs y Codigo/Herramientas/Audio/*.py
Estados del juego y guardado	Scripts/Managers/GameManager.cs
Teclado y control	Scripts/Sistema/Controles.cs

1. Cómo está armado el proyecto en Unity

¿Qué es un GameObject y cómo los creaste?

En Unity todo lo que hay en una escena es un **GameObject**, y lo que hace depende de los **componentes** que tiene: Transform (posición), SpriteRenderer (el dibujo), Rigidbody2D (la física), Collider2D (el choque) y mis scripts. Yo no arrastré los objetos a mano: los creo desde código con new GameObject() y AddComponent, y los guardo como **prefabs** (plantillas) con PrefabUtility.SaveAsPrefabAsset. Por ejemplo, Lira es un objeto con un hijo "Visual" que tiene el dibujo, y en el padre van el Rigidbody2D, el CapsuleCollider2D, Health, PlayerController y SpellCaster.

¿Por qué el dibujo va en un hijo "Visual"?

Para separar la física del dibujo. El collider se queda quieto en el padre y el hijo lo puedo estirar, aplastar, inclinar o voltear para animar a Lira sin mover su colisión.

¿Qué es un ScriptableObject?

Es un archivo de datos de Unity. Los uso para no meter números dentro del código: SpellData guarda el daño, costo de maná, recarga y velocidad de cada hechizo; ItemData lo que hace cada objeto; EnemyData la vida, daño y velocidad de cada enemigo. Si quiero balancear el juego cambio el archivo, no el script.

¿Cuáles son las escenas?

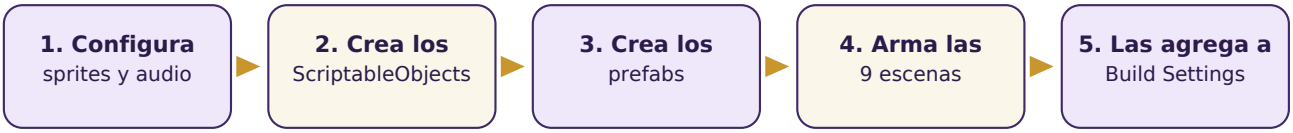
Son nueve, en este orden en Build Settings: Menú principal, Prólogo, las cinco alas (Aprendizaje, Fuego, Hielo, Viento y Corazón del Grimorio), Epílogo y Créditos. El **GameManager** pasa de una a otra con SceneManager.LoadScene y no se destruye al cambiar de escena (DontDestroyOnLoad).

¿Qué pilares de programación orientada a objetos usaste?

Pilar	Dónde lo uso
Herencia	EnemyBase es abstracta; EnemyAI, FlyingEnemyAI, RangedEnemyAI, ColosoAI y los demás heredan de ella y solo escriben su método Think(). Los jefes heredan de BossController.
Polimorfismo	Un hechizo trata a cualquier enemigo como EnemyBase y llama Stun() o GolpeDeHielo() sin saber si es un Golem o Elenora.
Abstracción	La interfaz IDamageable (TakeDamage) la implementa Health: la usan Lira, enemigos, muros y vasijas.
Encapsulamiento	Campos privados con [SerializeField] (se ven en el Inspector pero no se cambian desde otros scripts) y propiedades de solo lectura como IsGrounded.

2. Cómo creé los niveles

Respuesta corta: los niveles los programé. En CrearJuego.cs cada ala es una función (Nivel1() a Nivel5()) que coloca el suelo, las plataformas, los enemigos, los objetos y los diálogos con coordenadas. Con el menú **Ecos del Grimorio → Crear juego completo** se generan todas las escenas en un clic.



Así se ve una parte del Ala de Fuego en el código:

```
Suelo(n, -22, 15); Suelo(n, 18, 40);          // tramos de suelo (x inicial, x final)
Lava(n, 15, 18);                              // hueco con lava entre los tramos
Plat(n, 24, -2, 3);                            // plataforma en x=24, altura -2, ancho 3
Enemigo(n, "Centinela", 5, G, 2);              // enemigo con su zona de patrulla
Pagina(n, 65, 3.5f);                           // Página Perdida arriba de una plataforma
var kaelor = Jefe(n, "Kaelor", 116, G + 1.6f, 102, 130, 0);
Salida(n, 130, kaelor.GetComponent<Health>(), "Nivel3_AlaDeHielo", true, Elemento.Hielo);
```

¿Por qué no los armaste a mano en la escena?

- Si cambio algo del diseño solo vuelvo a correr el menú y las nueve escenas quedan iguales, sin errores de copiar y pegar.
- Todas las alas usan las mismas funciones (Suelo, Plat, Tunel, Escondite, Enemigo), así que agregar una sala nueva son pocas líneas.
- Me dejó adelantar el Ala de Viento y el Corazón del Grimorio, que estaban planeados para después.

¿Cómo le diste un estilo distinto a cada ala?

Cada ala tiene un **Tema** (clase Tema en CrearJuego): imagen de fondo, color de la piedra, del borde y de las plataformas, color de la luz, partículas de ambiente (polvo, brasas, nieve, hojas o motas) y qué adornos lleva. La misma textura de ladrillo se pinta del color de cada ala con el SpriteRenderer, por eso no necesité dibujar un tileset por nivel.

¿Cómo hiciste los secretos y los acertijos?

- **Sala secreta:** la función Escondite() pone una pared de piedra que tapa la sala y un muro agrietado con el script Rompible; al romperlo, la piedra se desvanece.
- **Pasadizo bajo:** Tunel() pone un techo bajo; Lira solo cabe agachada porque su collider se hace más chico.
- **Barreras de zarzas o hielo:** son Rompible con resistencia 0 a todo menos al Fuego.
- **Puertas rúnicas:** se abren cuando la caja (Empujable) llega a la placa de presión (PlacaPresion).

3. Los fondos y la profundidad

El fondo de cada ala tiene varias **capas** que se mueven a distinta velocidad cuando la cámara avanza. A eso se le llama **parallax** y da la sensación de profundidad. Lo hace mi script Parallax.cs:

```
Vector3 delta = cam.position - camStart;
transform.position = startPos + new Vector3(delta.x * factor, delta.y * factorVertical, 0);
// factor 1 = se queda pegado a la cámara (muy lejos), 0 = se mueve con el mundo
```

Capa (de lejos a cerca)	Qué tiene	Factor
Fondo pintado	La imagen del ala (dos copias, la segunda volteada).	0.88
Lejanas	Aurora (Hielo), estrellas (Corazón), nubes y aves (Viento), niebla alta.	0.55 a 0.85
Intermedias	Rayos de luz, columnas, humo de los hornos, nubes.	0.4 a 0.6
Cercanas	Estandartes, carámbanos del techo, niebla del suelo.	0.15 a 0.3
Nivel	Suelo, plataformas, libreros, cadenas, cristales, velas y libros flotantes.	0
Primer plano	Montículos oscuros abajo de la pantalla (se mueven al revés).	-0.25

Los adornos (rayos, niebla, estandartes, nubes, aves, libros, velas y copos de nieve) no son imágenes descargadas: los dibujo con código en EditorHelpers.cs, calculando el color de cada píxel con fórmulas (por ejemplo, el rayo de luz usa una curva de Gauss para que sea más fuerte al centro), y cada ala les da su color. Para que no se vean quietos uso Flotar.cs (suben y bajan), Deriva.cs (la niebla y las aves avanzan y regresan por el otro lado) y LuzParpadeante.cs (los hornos palpitan). La cara del suelo tiene un degradado oscuro para que no se vea plana.

4. Cómo hice que Lira y los enemigos queden pegados al suelo

Respuesta corta: el problema era que las imágenes de los personajes tienen espacio transparente abajo de los pies (en Lira es el 17 % del alto). El collider se calculaba con la imagen completa, así que los pies quedaban flotando. Hice una función que lee los píxeles del PNG, encuentra dónde empieza el dibujo de verdad y ajusta el collider y el detector de suelo a los pies reales.

Cómo funciona paso a paso

1. **Leer la imagen:** EditorHelpers.MargenesTransparentes() abre el PNG original con File.ReadAllBytes y Texture2D.LoadImage (la textura importada por Unity no se puede leer desde código).
2. **Buscar los píxeles visibles:** recorre todos los píxeles y guarda el más bajo, el más alto, el de más a la izquierda y el de más a la derecha cuyo alfa sea mayor a 20 (de 255).
3. **Calcular los márgenes:** devuelve cuánto espacio vacío hay abajo, arriba, a la izquierda y a la derecha, como porcentaje (por ejemplo 0.17 abajo).
4. **Ajustar el collider:** ParteVisible() calcula el tamaño y el centro de la parte dibujada. Con eso el CapsuleCollider2D de Lira empieza exactamente en sus pies.
5. **Mover el GroundCheck:** el círculo que revisa si Lira toca el suelo se coloca en los pies reales, no en la orilla de la imagen.
6. **Animación:** cuando Lira se aplasta al caer, PlayerController usa el campo margenPies para que el dibujo se encoja hacia sus pies y no hacia el centro.

```
// CrearJuego.cs - LiraPrefab()
ParteVisible(sprite, k, out size, out center);
float pies = center.y - size.y / 2f;
col.size = new Vector2(anchoImagen * 0.4f, size.y * 0.97f);
col.offset = new Vector2(center.x, pies + col.size.y / 2f);
groundCheck.localPosition = new Vector3(center.x, pies, 0f);
```

Enemigos y personajes: la función EnemyBody() que crea a todos los enemigos usa la misma ParteVisible(), y los personajes con diálogo (Maestra Sable, espíritus) se bajan lo que mide su margen transparente.

Otras cosas que ayudan a que no se “resbalen” o se atoren

- **Rigidbody2D** con freezeRotation (no se voltean) y detección Continuous (no atraviesan el suelo al caer rápido).
- **CapsuleCollider2D:** la cápsula tiene las esquinas redondas y no se atora en la orilla de las plataformas.
- **Material sin fricción** (SinFriccion.physicsMaterial2D): Lira no se queda pegada a las paredes al saltar contra ellas.
- **Detección del suelo:** CheckGround() usa Physics2D.OverlapCircleAll en los pies y solo cuenta objetos con el tag “Ground”; las paredes no lo tienen, por eso no se pueden

escalar.

- **Sombra:** un óvalo oscuro debajo de Lira que se achica cuando está en el aire; ayuda a ver dónde va a caer.

5. El movimiento de Lira

- **Correr con aceleración:** la velocidad sube poco a poco con Mathf.MoveTowards, por eso se siente suave.
- **FixedUpdate:** la física se calcula 50 veces por segundo, separada de los cuadros por segundo.
- **Coyote time (0.1 s):** puedes saltar un instante después de salir de la orilla.
- **Búfer de salto (0.12 s):** si presionas saltar justo antes de tocar el suelo, igual salta.
- **Salto variable:** si sueltas el botón antes, la velocidad hacia arriba se corta a la mitad.
- **Gravedad mayor al caer** (x1.6) para que el salto se sienta firme; al planear con Viento baja al 25 %.
- **Esquive:** un impulso de 0.18 s en el que Lira es invulnerable (Health.SetInvulnerable).
- **Agacharse:** el collider baja al 60 % de su altura para pasar por los túneles.
- **Piernas animadas por código:** el sprite se corta en cuerpo y dos piernas con Sprite.Create y las piernas giran desde la cadera al caminar. No dibujé animaciones cuadro por cuadro.
- **Giro:** al cambiar de dirección la escala en X pasa por 0 en unos cuadros, así se ve que se da la vuelta.

6. Los hechizos y los combos

Respuesta corta: cada hechizo es un ScriptableObject SpellData con su daño, costo, recarga, velocidad y efecto. SpellCaster tiene tres espacios; al presionar J, K o L (o Cuadrado, R1, L1) revisa el maná y la recarga y crea el proyectil. El proyectil (SpellProjectile) avanza con un Rigidbody2D y al chocar hace daño con el elemento del hechizo.

Hechizo	Daño	Maná	Recarga	Extra
Arcano	15	10	0.3 s	Rompe vasijas y muros agrietados.
Fuego	25	18	0.6 s	Quema zarzas y derrite hielo.
Hielo	10	12	0.5 s	Ralentiza; al tercer golpe seguido congela.
Viento	8	12	0.5 s	Empuja enemigos y cajas; deja planear.

¿Cómo funcionan los combos?

Si lanzo **dos elementos distintos en menos de 0.9 segundos**, el método GetCombo() revisa la pareja y en lugar del segundo hechizo sale el combo. No importa el orden. Cada combo cuesta 15 de maná.

```
if (Has(Elemento.Arcano) && Has(Elemento.Fuego)) return TipoCombo.ExplosionArcana;
if (Has(Elemento.Fuego)  && Has(Elemento.Hielo))  return TipoCombo.VaporCegador;
if (Has(Elemento.Hielo)  && Has(Elemento.Viento)) return TipoCombo.GranizoCortante;
if (Has(Elemento.Fuego)  && Has(Elemento.Viento)) return TipoCombo.TormentaDeAscuas;
```

Combo	Qué hace en el código
Explosión Arcana	AreaEffect.Damage: 40 de daño en un círculo de 2.5 unidades frente a Lira.
Vapor Cegador	Aturde (Stun) 2.5 s a todos los enemigos a menos de 3 unidades.
Granizo Cortante	Proyectil de hielo que atraviesa enemigos (pierce) con 2.5 veces el daño.
Tormenta de Ascuas	Corrutina: tres golpes de fuego alrededor de Lira, uno cada 0.4 s.

¿Cómo se apunta?

Por defecto el hechizo sale al frente. Con W (o el stick arriba) sale hacia arriba o en diagonal, con el stick derecho en cualquier dirección (360°) y con el clic hacia el mouse. La **ayuda para apuntar** busca enemigos vivos a menos de 11 unidades y a menos de 50° de la mira, sin pared en medio (Raycast); el proyectil gira hasta 320° por segundo para seguirlo. Se puede quitar en Opciones.

¿Cómo funcionan el maná y la vida?

El maná se recupera solo (9 por segundo) y con los Cristales de Maná. La vida está en Health: recibe el daño con el elemento, lo multiplica por la resistencia del objetivo (Kaelor resiste el fuego con 0.5 y recibe 1.5 de hielo). A Lira además le da 1.5 segundos de invulnerabilidad después de cada golpe y recibe 35 % menos daño.

7. Las habilidades de cada elemento

Habilidad	Cómo la programé
Fuego quema zarzas y derrite hielo	La barrera es un Rompible con resistencia 0 a todos los elementos menos al Fuego. Si le pegas con otro, Health lanza el evento Resisted y la barrera muestra una pista.
Hielo congela	EnemyBase.GolpeDeHielo() cuenta los golpes; al tercero en menos de 4 s el enemigo queda aturdido 2.5 s y se pinta de azul.
Viento empuja	Empujable.cs: el proyectil de Viento le da velocidad a la caja. Cuando la caja toca la PlacaPresion, la PuertaRunica se abre. Si la caja se cae, regresa a su lugar.
Planear	Con Viento aprendido, mantener saltar mientras caes baja la gravedad al 25 % y limita la velocidad de caída.
Corrientes de aire	CorrienteViento.cs es un trigger que empuja hacia arriba a Lira mientras está dentro.
Agacharse	Pasa por túneles y el hechizo sale a ras de suelo para los enemigos pequeños.

8. Enemigos, jefes y armas

Enemigos: todos heredan de EnemyBase, que ya tiene vida, daño por contacto, empuje, ralentización, aturdimiento, barra de vida y botín. Cada tipo solo escribe su comportamiento en Think(), que se llama en cada cuadro:

Clase	Comportamiento	Enemigos
EnemyAI	Patrulla entre dos puntos y persigue a Lira si la ve.	Espectros, Centinelas, Salamandras
FlyingEnemyAI	Flota alrededor de su punto; tiembla como aviso y se lanza en picada.	Aves, Golems, Motas
RangedEnemyAI	Se queda quieto y dispara proyectiles.	Cristales Vivientes
ColosoAI / GargolaAI	Pisotón en área / estatua que despierta.	Coloso de Raíz, Gárgola
BossController	Fases por porcentaje de vida y ataques propios.	Kaelor, Isolde, Threnody, Elenora

¿Cómo funcionan las armas de los enemigos?

ArmaEnemigo.cs se agrega al enemigo con el dibujo de su arma (Garras de Tinta, Lanza Incandescente o Embestida Rocosa). Cuando Lira está dentro del alcance, el arma aparece frente al enemigo, espera un **aviso** de 0.35 s (para que puedas esquivar) y daña lo que esté en esa zona. Cada arma tiene su tipo de golpe: zarpazo, estocada o embestida, su daño y su recarga.

¿Cómo funcionan los jefes?

BossController revisa la vida: al bajar de cierto porcentaje (por ejemplo 50 %) cambia de fase y se vuelve más agresivo. Cuando el jefe muere lanza el evento BossDefeated: el AudioManager regresa la música del nivel y el AchievementManager revisa los logros, sin que el jefe los conozca. Elenora tiene cuatro fases y cambia de elemento en cada una.

9. El truco: Modo Archimaga

Es el código secreto del documento de diseño. Con el teclado se escribe **F, H, V, A** y **Enter**; con el control se presionan **L1 + R1 + Triángulo** al mismo tiempo. Durante 60 segundos Lira tiene maná infinito y sin espera entre hechizos.

```
private readonly Key[] code = { Key.F, Key.H, Key.V, Key.A, Key.Enter };
private int progress;   // cuántas teclas del código van bien

// en Update: si la tecla que presionó es la que sigue, avanza;
// si se equivoca o tarda más de 4 segundos, vuelve a empezar.
// cuando progress llega al final -> caster.SetArchmageMode(true) por 60 s
```

Mientras está activo, SpellCaster.SpendMana() regresa verdadero sin gastar maná y no revisa la recarga. Al activarlo por primera vez se desbloquea el logro “Secreto Revelado”.

10. La música y los sonidos

Respuesta corta: la música, los efectos y las voces los generé con scripts de **Python** (NumPy y SciPy) que crean las ondas de sonido con matemáticas, y en Unity un solo objeto, el **AudioManager**, los reproduce todos.

Cómo se genera el sonido

- **Instrumentos:** cada instrumento es una función. Por ejemplo, una campana es una onda seno modulada en frecuencia (FM) que se apaga con una curva exponencial; las cuerdas son ondas diente de sierra con un filtro pasa bajas.
- **Música:** en musica.py escribo los acordes y la melodía con nombres de notas (C4, E4, G4…) y el script los convierte en frecuencias, los mezcla y les pone reverberación. Son 9 pistas (menú, una por ala, jefe, jefe final y créditos) que se repiten en bucle; se convierten a OGG con FFmpeg.
- **Efectos:** en efectos.py cada sonido es una función: el salto es un barrido de frecuencia que sube, la explosión es ruido filtrado, el latido son dos golpes graves de 70 y 60 Hz. Son 41 efectos.
- **Voces:** cada personaje tiene 8 sílabas cortas con su propio tono; mientras aparece el texto del diálogo suena una al azar.

Cómo suena dentro de Unity

- El **AudioManager** va dentro del prefab del GameManager, que no se destruye al cambiar de escena, así que la música no se corta.
- Usa el patrón **Singleton**: cualquier script hace AudioManager.Play(Sfx.Salto) sin buscar el objeto.
- Los efectos están en un enum Sfx en el mismo orden que los archivos; CrearJuego los conecta solos por su nombre.
- Para cambiar de música hace un **fundido** (crossfade) con dos AudioSource: uno baja mientras el otro sube.
- La música usa **Streaming** (se lee del disco mientras suena) para no ocupar memoria; los efectos cortos se cargan completos para sonar al instante.
- Hay 10 canales para efectos y un límite para que el mismo sonido no se repita en el mismo instante.

11. Otros sistemas

Sistema	Cómo funciona
Estados del juego	GameManager tiene un enum GameState (MenuPrincipal, Jugando, Pausa, Logros, GameOver, NivelCompletado, Creditos) y un evento StateChanged que escucha la interfaz.
Guardado	PlayerPrefs guarda el ala actual, hechizos, fragmentos, páginas, objetos recogidos, logros y opciones.
Controles	Controles.cs junta el teclado y el control con el New Input System y detecta cuál se usó al último para mostrar los botones correctos.
Logros	AchievementManager escucha eventos (hechizo aprendido, jefe vencido, combo, nivel terminado) y guarda los 10 logros.
Interfaz	UGUI con Canvas Scaler a 1920×1080, fuente Poppins Bold y marcos 9-slice que no se deforman.
Luces	Universal Render Pipeline 2D con Light2D: antorchas, hornos, cristales, velas, portales y la luz que acompaña a Lira.
Sentido arcano	SentidoArcano.cs mide la distancia de Lira a páginas, fragmentos y muros secretos y muestra chispas, un tintineo y un aviso.
Saltar nivel	Mantener N o L3 + R3 1.5 s (o Pausa → Saltar nivel) llama al portal del ala, que da el hechizo y carga el siguiente nivel; no cuenta para logros.
Ejecutable	Menú Ecos del Grimorio → Compilar juego (.exe): usa BuildPipeline.BuildPlayer para Windows de 64 bits y guarda todo en la carpeta Build.

12. Preguntas rápidas

Respuestas de una o dos líneas para cuando me pregunten algo directo.

Pregunta	Lo que respondo
¿En qué lo hiciste?	Unity 6 con Universal Render Pipeline 2D, programado en C# con Visual Studio Code.
¿Cuántos scripts tiene?	67 del juego y 4 herramientas de editor.
¿Cómo hiciste los niveles?	Con una herramienta de editor que programé (CrearJuego.cs) que arma las escenas desde código.
¿Por qué ya no flota Lira?	Porque el collider se calcula con la parte visible del dibujo, leyendo los píxeles transparentes del PNG.
¿Cómo detecta el suelo?	Con un círculo en los pies (OverlapCircle) que solo cuenta objetos con el tag Ground.
¿Dónde están los datos de los hechizos?	En ScriptableObjects SpellData: daño, maná, recarga y velocidad.
¿Cómo salen los combos?	Dos elementos distintos en menos de 0.9 s; GetCombo() revisa la pareja.
¿Cómo hiciste los enemigos?	Con herencia: todos heredan de EnemyBase y solo cambian su método Think().
¿Cómo funciona el truco?	Un arreglo con las teclas F, H, V, A, Enter y un contador que avanza si presionas la que sigue.
¿De dónde es la música?	La generé con scripts de Python que sintetizan las ondas; en Unity la reproduce el AudioManager.
¿Cómo no se corta la música?	El AudioManager no se destruye entre escenas y hace un fundido con dos AudioSource.
¿Cómo es el fondo con profundidad?	Parallax: cada capa se mueve a una fracción de la velocidad de la cámara.
¿Cómo se guarda la partida?	Con PlayerPrefs al terminar cada ala y al recoger objetos.
¿Y el arte?	Bocetos a lápiz, arte digital con Copilot Pro a partir de los bocetos y limpieza en GIMP; los adornos del escenario los dibujé con código.
¿Cómo hiciste el .exe?	Con un menú que programé que llama a BuildPipeline.BuildPlayer para Windows.